

CHRONOVAULT

Timelock Encryption System

Security Architecture & Documentation

Document Version	2.1
Date	22.12.2025
Classification	Technical Documentation

1. Executive Summary

ChronoVault is a client-side timelock encryption application that combines traditional password-based encryption with time-based unlocking mechanisms powered by the drand distributed randomness beacon.

The system protects sensitive data with two independent security layers: a user-defined password and a time-based lock that cannot be bypassed until a specific future time has been reached.

Key Security Properties

- AES-256-GCM encryption for data confidentiality and integrity
- PBKDF2 key derivation with 100,000 iterations
- Distributed timelock via drand League of Entropy
- Zero server-side storage — fully client-side operation
- Dual-layer encryption (password + timelock)

2. Data Privacy & Network Communication

✓ USER DATA NEVER LEAVES THE COMPUTER

All encryption and decryption operations are performed entirely within the user's web browser using the Web Cryptography API. No user data is ever transmitted to any server.

What Stays Local (Never Transmitted)

- Your secret text or PDF content
- Your password
- Encryption keys (derived locally)
- The encrypted vault file
- All cryptographic operations

Network Communication (drand API Only)

The only network communication is with the public drand API (api.drand.sh). These requests:

- **Contain ZERO user data**
- Only fetch the current round number (a public timestamp)
- Are read-only GET requests
- Could be cached or proxied without security impact

Example drand Request

```
GET https://api.drand.sh/52db9ba.../public/latest
Response: { round: 14523456, randomness: "abc123..." }
```

This request contains no user information and the response is public randomness data available to anyone.

3. Security Architecture

Dual-Layer Encryption Model

ChronoVault implements a dual-layer encryption scheme providing both password protection and timelock functionality.

Layer	Protection Type	Key Derivation	Purpose
Layer 1 (Inner)	Password-Based	PBKDF2-SHA256, 100K iterations	User authentication
Layer 2 (Outer)	Timelock	PBKDF2 + Round ID	Time-based access

Client-Side Security Model

All cryptographic operations occur within the browser using the Web Cryptography API (`window.crypto.subtle`). Benefits:

- **No server trust required:** Private keys and plaintext never leave the client
- **Hardware-backed RNG:** Cryptographic random numbers from OS/hardware
- **Constant-time operations:** WebCrypto resists timing attacks

4. Cryptographic Standards

ChronoVault employs industry-standard cryptographic algorithms recommended by NIST.

Component	Algorithm	Standard	Parameters
Symmetric Encryption	AES-256-GCM	NIST SP 800-38D	256-bit key, 96-bit IV
Key Derivation	PBKDF2	NIST SP 800-132	SHA-256, 100K iter
Hashing	SHA-256	FIPS 180-4	256-bit output
Random Generation	CSPRNG	NIST SP 800-90A	OS/Hardware entropy

5. drand Integration

The drand network is a decentralized randomness beacon operated by the League of Entropy, including Cloudflare, EPFL, Kudelski Security, and Protocol Labs.

How drand Provides Timelock

1. drand produces verifiable random values ("rounds") at fixed 3-second intervals
2. Each round has a deterministic timestamp based on genesis time + (round × period)
3. Future round numbers can be calculated, but values are unknown until produced
4. ChronoVault calculates which round corresponds to the unlock time
5. Decryption requires the round to exist (time passed) for verification

drand Configuration

Property	Value	Security Implication
Network	quicknet	Production-grade, high availability
Round Period	3 seconds	Fine-grained time resolution
Threshold	Byzantine fault tolerant	No single point of failure
Verification	BLS signatures	Cryptographically verifiable

6. Security Features

Password Verification (v2.1)

ChronoVault v2.1 implements a slow password verifier to provide brute-force resistance and enable early password verification before the timelock expires.

- **Password Hash:** SHA-256(password + base64(salt)) - stored for reference
- **Password Verifier:** HMAC-SHA256(PBKDF2-derived key, constant) with 200,000 iterations
- **Early Verification:** Wrong passwords are rejected immediately, before waiting for timelock
- **Separate Salt:** The verifier uses a dedicated 256-bit salt (verifierSalt) independent from the encryption salt

Timelock Feedback System

Condition	User Feedback
Wrong password, time not reached	Password incorrect. Cannot be revealed before [date/time]
Correct password, time not reached	Password correct! Cannot be revealed before [date/time]
Wrong password, time reached	Password incorrect. Vault unlockable but wrong password
Correct password, time reached	Success — data decrypted and returned

Additional Security Measures

- **Unique salts:** Every encryption uses a fresh 256-bit random salt
- **Unique IVs:** 96-bit initialization vectors generated per encryption
- **No key reuse:** Each layer uses independently derived keys
- **Authenticated encryption:** GCM mode detects tampering

7. Risks & Limitations

⚠ IMPORTANT: Users should understand these limitations before relying on ChronoVault for critical data protection.

Critical Risks

Risk	Severity	Description	Mitigation
Password Loss	CRITICAL	Lost password = permanent data loss. No recovery.	Use password manager
Vault File Loss	CRITICAL	Deleted/corrupted .vault file cannot be recovered.	Keep multiple backups
drand Network Failure	HIGH	If drand permanently fails, timelocked data may be inaccessible.	Decentralized design
Browser Vulnerabilities	MEDIUM	Malicious extensions could intercept data.	Use trusted browser
Weak Password	MEDIUM	Short passwords vulnerable to brute-force.	Use 12+ characters

Implementation Limitations

- **Not true timelock cryptography:** Uses drand round availability as proxy for time, not cryptographic timelock puzzles
- **Client-side only:** No server-side enforcement. Sophisticated attacker could modify JavaScript
- **10MB file limit:** Large files may cause browser memory issues
- **No forward secrecy:** Compromised vault with correct password reveals data once time passes

8. Best Practices

For Users

1. Use strong, unique passwords (minimum 12 characters with mixed case, numbers, symbols)
2. Store passwords in a reputable password manager
3. Keep multiple backups of .vault files in different locations
4. Verify the unlock date/time carefully before encrypting
5. Use a clean, trusted browser without suspicious extensions
6. Do not share vault files over insecure channels

For Deployment

1. Serve over HTTPS only
2. Implement Content Security Policy (CSP) headers
3. Use Subresource Integrity (SRI) for external resources
4. Regular security audits of the codebase

9. Technical Specifications

Vault File Format

The .vault file is a JSON document containing encrypted data and metadata:

```
{
  {
    "version": "2.1",
    "type": "text|pdf|file",
    "timelock": { "targetRound": 12345, "unlockTime": "ISO-8601", ... },
    "crypto": {
      "algorithm": "AES-256-GCM", "kdf": "PBKDF2",
      "salt": "base64...", "iv": "base64...", "dataIv": "base64...",
      "verifierSalt": "base64...", // NEW in v2.1
      "passwordVerifier": "base64...", // NEW in v2.1
      "passwordHash": "base64..." // SHA-256(password + base64(salt))
    },
    "data": "base64-encoded-encrypted-data"
  }
  "timelock": { "targetRound": 12345, "unlockTime": "ISO-8601" },
  "data": "base64-encoded-encrypted-data"
}
```

Supported Browsers

Browser	Minimum Version	Web Crypto Support
Chrome/Edge	37+	Full
Firefox	34+	Full
Safari	11+	Full

— End of Document —